Hyper-parameter Tuning of Federated Learning Based on Particle Swarm Optimization

Zhiyuan Li[1], Hao Li[1*], Mingyang Zhang[1]

[1]School of Electronic Engineering, Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education of China, Xidian University, Xi'an 710071, China

2021 IEEE 7th International Conference on Cloud Computing and Intelligent Systems (CCIS) | 978-1-6654-4149-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/CCIS53392.2021.9754676

Abstract: The learning task of federated learning (FL) is solved by a federation of a center server and individual clients. Contrary to traditional deep learning models, federated learning consists of two parts, the global model and individual models. However, since the process is carried out by two parts of equal importance, it is ideal to deal with both parts simultaneously. To achieve superior performance, federated learning requires carefully selected hyper-parameters, which has more hyper-parameters than those of traditional deep learning models. To solve this tuning problem, we propose a method using particle swarm optimization (PSO) algorithm to tune the hyperparameters of federated learning. PSO algorithm is a gradient-free, stochastic optimization method which is better than grid search method when it comes to large search space. It helps locate the optimal combination of multiple parameters. In this article, we focus on applying PSO method on tuning the hyper-parameters of FL models, and prove that it is an efficient way to acquire satisfactory results. Experiments on MNIST dataset with convolution neural networks have proved the superiority of the proposed method.

Keywords: Federated learning; Hyper-parameter tuning; PSO algorithm

1 Introduction

Federated learning (FL) is first introduced in 2016 for users to share information by training a global model and update it collaboratively [3]. The unique feature about federated learning is that instead of processing centralized data, it deals with separated data and only updates parameters sent by individual users. Due to this character, FL has aroused great interest in the field of healthcare, autonomous vehicles and mobile device monitoring, where privacy is highly valued [4].

The structure of the FL consists of two parts, the global model and individual clients to share parameter update. The hyper-parameters of both parts are determining-factors on the performance of FL. The parameters such as the neurons in hidden layers, learning rate and momentum affects the performance of the global model and also how well they can communicate with the individual clients about parameter update. The communication is determined by the fraction of users activated, training epochs on individual clients and the number of updates.

978-1-6654-4149-0/21/$31.00 ©2021 IEEE

Hyper-parameters tuning has remained a problem for FL for the following reasons: (i) The potential hyper-parameters of clients added to existing ones made tuning process complex and expensive. (ii) Accuracy is not the only factor to look into, the communication between global model and clients can be unreliable due to unavailable devices, getting a good trade-off between accuracy and convergence requires a wide search when it comes to larger search space [1]. In general, FL is constrained by clients' conditions, thus its tuning process needs to meet some of the requirements such as the limited calculate ability on the individual devices, getting accessible information when updating and reducing the cost in communication. A way to satisfy all these requirements while getting high accuracy is wanted.

This article proposes a new method to address these problems based on particle swarm optimization (PSO). With PSO algorithm being an easy to implement, robust and efficient way to optimize complicated problems [5], it is useful to deal with optimization task with large searching space. Also, its updating algorithm saves a lot of computer resources compared to traditional grid search method. Even though PSO algorithm does not guarantee an optimal solution, its random select of parameters gives a way to jump out of local optimal point. Genetic algorithm (GA) is a traditional evolution algorithm. Compared to PSO, it is more reliable and easy to implement, however, it converges slower and is easy to converge to local optimal point. This article included both methods to compare their performance on FL hyper-parameters tuning.

2 Related work

A lot of work has done on the hyper-parameter tuning of deep-learning model, most of them focus on the accuracy [6] since it is the most crucial factor when evaluating deep learning models. PSO based parameter optimization has already been implemented in many occasions, Shi et al. [8] has applied PSO to choose proper kernel for SVM. Qolomany et al [2] has proven that PSO is also useful when tuning the hyper-parameters of deep learning models such as LSTM. Both of the articles chose two hyper-parameters in the process.

With FL being a relatively new proposed model, few have looked into its tuning process. Accuracy is not the

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:39:12 UTC from IEEE Xplore. Restrictions apply.

only goal to pursuit, how generalized the global model is also a critical factor.

Some work was done and focused on the global model accuracy, while a recent study has focused on the tuning process of individual clients [2]. The model performance on individual models is important but often overlooked, the training epoch decides how specialized the individual model is. The individual accuracy however, does not directly guarantee a better performance for global model. It can be too specialized and not able to generalize the global model. Also, some individual models don not participate in the updating process.

The method in this article look into the tuning process combining the global hyper-parameters with individual hyper-parameters since models of different structure with similar accuracy can be specialized differently, thus result in different respond to update process.

3 Methodology

The method applied PSO in the tuning of both global parameters and individual parameters of FL. The parameters looked into are neurons in hidden layer(n), learning rate(η), momentum(m) of the global model, training epoch(e) of the individual model and fraction(c) of clients activated in each round of update.

3.1 Structure of the FL model

The character of dataset in FL is different from classic deep-learning problems. Most of the networks deal with IID dataset while FL environment deals with non-IID dataset since the information from different clients can be unbalanced. The optimization method in this article is FedAvg proposed by Mcmahan et [3]. It is proven to guarantee convergence on non-IID dataset [7]. In this article we seperate MNIST dataset to make it non-IID.

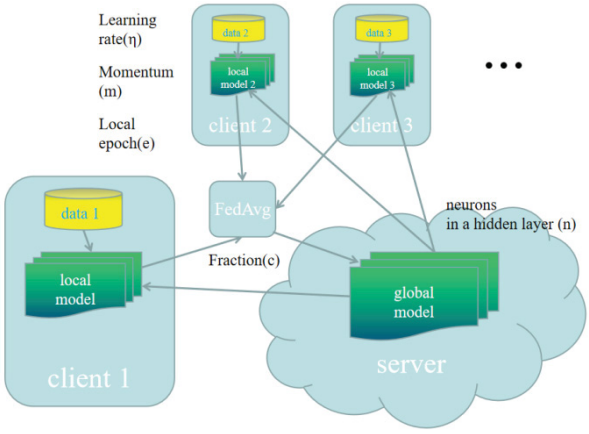


Figure 1 Illustration of the federated learning with parameters to be tuned. The global model is stored in server while local models were trained with individual dataset. Then the global model is updated with FedAvg algorithm, as is shown in Algorithm I

CNN is used as the deep learning model to evaluate proposed method, it gives great performance with simple structure. In this case, a simple neural network with one hidden layer is enough to give good performance.

Global model is first initialized, share same parameters with individual models and then constantly updated by individual models. Each individual client train a local model, then send the change to server, where global model gets updated. The communication process is shown in Figure 1. In this experiment we update the global model 10 times and then test its accuracy on testing set. As is shown in Algorithm I.

ALGORITHM I ALGORITHM FOR FEDAVG

Input: number of clients K, local mini-batch size B, local epochs E, learning rate η, global epoch T, update fraction c
Initialize the weights in clients and the server
for $t = 1,2,...,T$:
  Randomly select $K = [c*K]$ clients
  for $k = 1,2,...,N$:
    $w_{t+1}^k \leftarrow \mathbf{Local_Update}(B, E, \eta, w_t^k)$
  $w_{t+1} \leftarrow \sum_{k=1}^{K} \frac{n^k}{N} w_{t+1}^k$
Local_Update(B, E, η, w_t^k):
  Local model updates w_t^k with B mini-batch size for E epochs with η.
  Return w_{t+1}^k

MNIST dataset is first divided into many shards, each shard has the same label, then these shards are assigned to each client. In this way, the dataset is set to be non-IID.

3.2 Structure of the proposed PSO method

PSO was originally inspired by the behavior of bird swamp, they share information on their location and speed to help group hunting and avoiding predators. PSO algorithm has gained a lot of attention for its easy implement, robustness and fast convergence.

PSO algorithm is initialized to be a swamp of particles representing models with different parameters, each particle is assigned with initialized location, velocity and fitting value. They are designed to communication with each other and update their velocity and location according to the fitting value. In each iteration, particles follow two well-performed particles, the *gbest*, global best particle that has best fitting value over all particles over all iterations, and the *pbest*, the best fitting value the certain particle got in the process.

The position of the particle is updates by its velocity, according to equation(1):

$$L_i^{t+1} = L_i^t + V_i^t \tag{1}$$

Where L_i^t stands for the location of i-th particle at iteration t. V_i^t stands for the velocity of i-th particle at iteration t. Particles are trying to get closer to global and historical optimal, thus the velocity update is given by equation(2):

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:39:12 UTC from IEEE Xplore. Restrictions apply.

$$V_i^{t+1} = w \cdot V_i^t + c_1 \cdot rand \cdot (L_{gbest}^t - V_i^t) + c_2 \cdot rand \cdot (L_{i,t}^{pbest} - V_i^t) \quad (2)$$

Where L_{gbest}^t stands for the location of global best particle at iteration t. $L_{i,t}^{pbest}$ stands for the location of best performing i-th particle at iteration t. c_1 and c_2 are the learning coefficients deciding the how fast particles move towards best performs particles. w is the inertia coefficient that helps finding optima, bigger w leads to a better performance in finding the global optima. To improve performance, w is modified to change through iterations so that the algorithm can search faster initially and slow down in deep iterations to converge to optima.

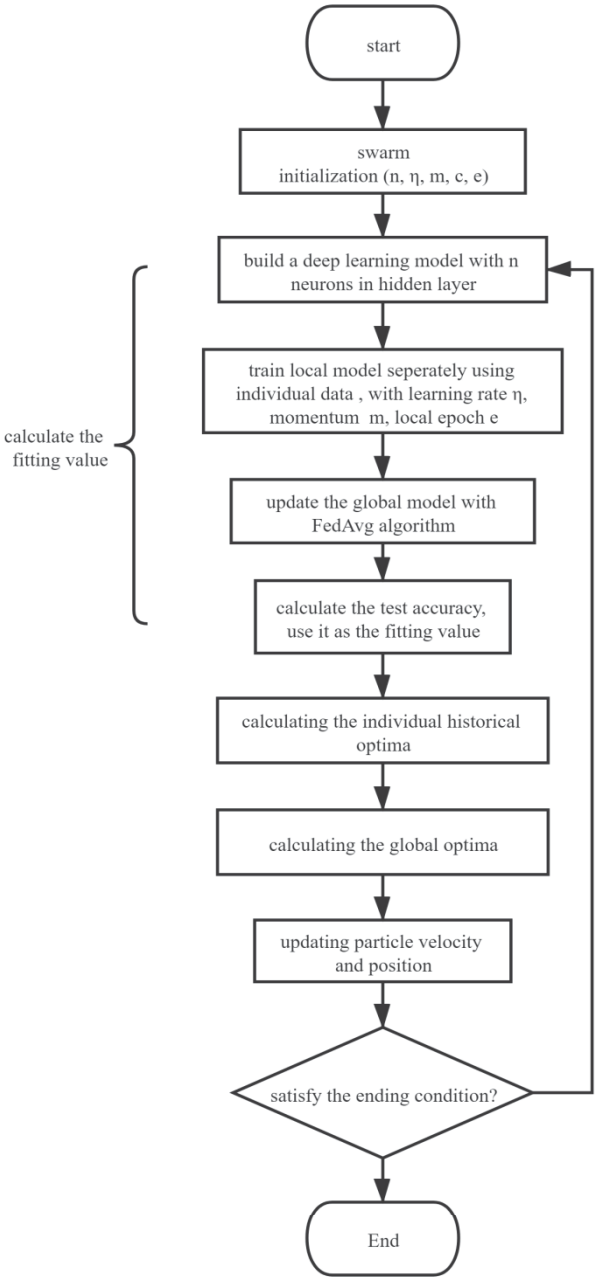


Figure 2 flowchart of the particle swarm optimization algorithm in FL environment, the fitting value is the testing accuracy of the global model on the testing set

Table I Parameters utilized in the experiment

parameters	value
Number of neurons(n)	**32**
Learning rate(η)	**[0.001,0.021]**
Momentum(m)	**Within the range [0.1, 0.9]**
Fraction(c)	**Within the range [0.1, 0.3]**
Local epoch(e)	**Within the range [3, 7]**
Coefficient c1,c2	**2**
Number of clients	**100**
Number of epochs	**10**
Local mini-batch size	**10**
c1	**2.0**
c2	**2.0**
w	**[0.4, 0.9]**

ALGORITHM II PSO FOR HYPER-PARAMETER TUNING IN FL

Input: dataset, bounds for article value, coefficients (c1, c2, w), number of iteration T
Output: optimal model in terms of the selected parameter for FL
Initialize the velocity of each particle with the same length as particle location, the location of each particle, randomly distributed within the bound of each parameter
Define the fitting value, the parameters of the particle was then tested in FL with FedAvg accordingly, the testing accuracy was then used as the fitting value.
for t = 1,2,...,T:
    get fitting value of each particle
    Update global best and historical best
    Update the particle location following equation
Return the optimal particle parameters for the FL model
End

The parameters used in the algorithm is listed in table I, first five parameters are the parameters to be optimized, thus the location of a particle is [n, η, m, c, e]. These parameters get constantly updated in the process as is shown by a flowchart in Figure 2.

Velocity of the particle then searches through the given range. As is shown in Algorithm II.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:39:12 UTC from IEEE Xplore. Restrictions apply.

4 Experimental studies

4.1 experimental settings

In the experiment we selected genetic algorithm (GA) to compare the performance with that of the proposed PSO algorithm. We train CNN with one hidden layer on MNIST dataset, the number of clients is assigned to 100, MNIST is then divided into 200 different shards according to their labels, each client consists of 2 shards to stimulate the non-IID environment.

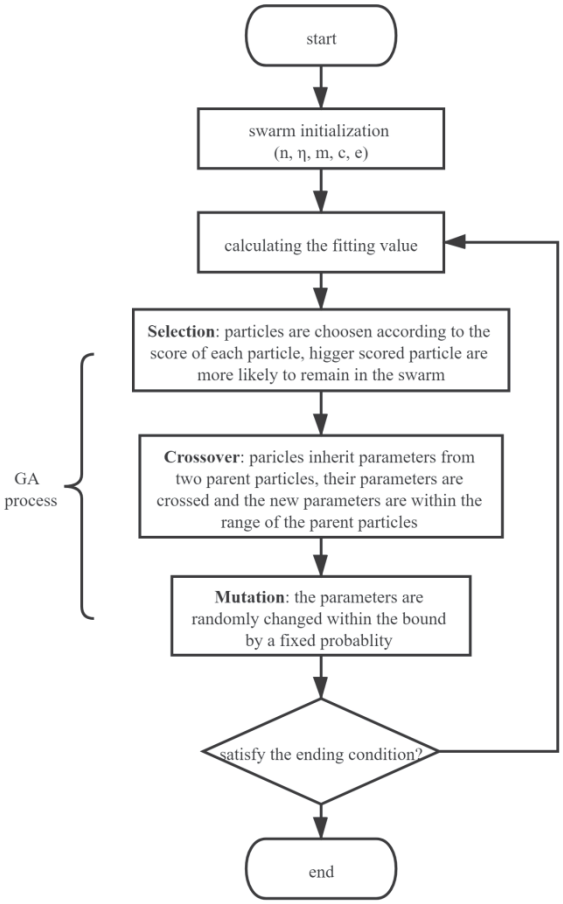


Figure 3 flowchart of the genetic algorithm, the fitting value is calculated the same way as is done in PSO algorithm shown in figure 2.

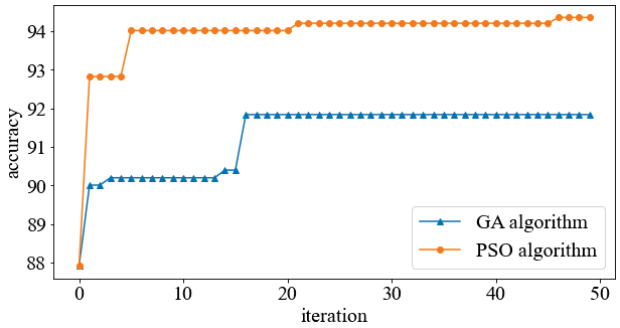
Number of neurons in a model determines how specialized the global network is. In FL, local epoch determines how specialized a local model is and fraction of the client indicates the fraction of clients participate in each update process, global model gets updated by FedAvg that randomly selects clients by a fixed ratio of total clients. Only the selected clients were included in the updating process. Together with learning rate and momentum, these parameters collaborate and decide how specialized local and global models are, how global model communicates with local models and together how they perform on the given dataset.

To evaluate the performance of PSO algorithm, GA algorithm and grid search algorithm are implemented to compare. The flowchart of the algorithm shows how swarm evolve and mutate to come close to an answer, as is shown in Figure 3.

4.2 experimental results

In the experiment, both algorithms are initialized with the same swarm for comparison, as shown in figure 4. The settings of the swarm are 8 particles each iteration and 50 iterations total. PSO algorithm is compared with GA and grid search. Both PSO and GA start with same swarm with best accuracy being 87.9%. PSO algorithm converges faster and reaches a better solution than that of GA.

Both algorithms are more efficient than grid search. Shown in figure 5 is the comparison of PSO algorithm and grid search, one iteration for grid search means the number of particles searched is the same as the particles in one PSO iteration (8 in this case). The setting of grid search is that it traverses through the search space



starting with less neurons in hidden layer.

Figure 4 comparison between GA and PSO in terms of the accuracy of optimal particle

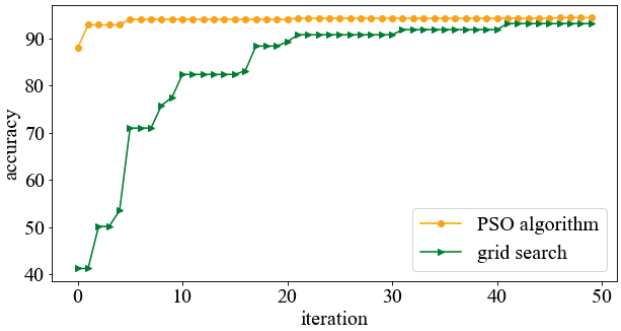


Figure 5 comparison between PSO and grid search in terms of the accuracy of optimal particle

Even though the accuracy of the grid search reached 93.1%, it can not be improved without much more computation, since the scale of a swarm can be modified but the grid search needs to explore the entire search space.

Different initialization can lead to different optimizing process, in PSO algorithm, swarm with bad initialization result can still can decent results, as is shown in figure 6. The solution gained was 90.3% and 94.3% respectively.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:39:12 UTC from IEEE Xplore. Restrictions apply.

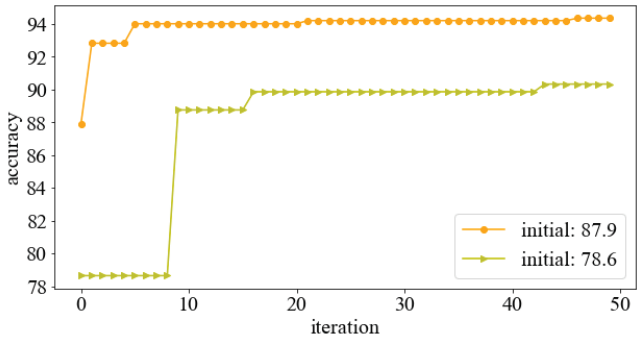


Figure 6 comparison between PSO algorithm with different initial swarm in terms of the accuracy of optimal particle

The optimization process is not only affected by initialization, it is also largely decided by the swarm scale, number of iteration and particle dimension. To gain better performance, a different setting of PSO algorithm is implemented. As is shown in figure 7 and figure 8, a PSO with 32 particle each iteration reached final accuracy of 95.2%.

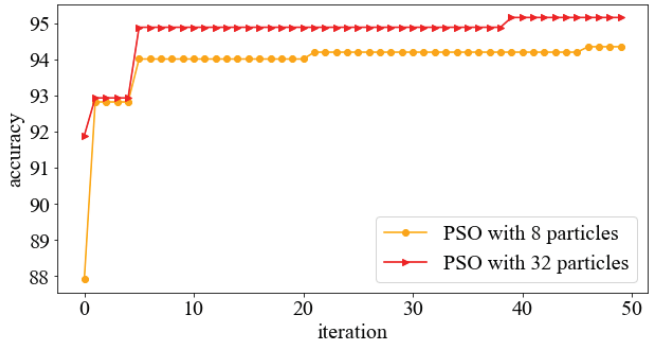


Figure 7 comparison between PSO algorithm with different particles in each iteration in terms of the accuracy of optimal particle

Table II comparison between PSO algorithm with different particles regarding their optimizing process

parameters	8p PSO	32p PSO
Particles in an iteration	**8**	**32**
Initial accuracy(%)	**87.9**	**91.9**
Accuracy after 10 iterations(%)	**94.0**	**94.9**
Accuracy after 25 iterations(%)	**94.2**	**94.9**
Accuracy after 50 iterations(%)	**94.4**	**95.2**

As is shown in the figure, more particles lead to a better performance, when the global accuracy reached a certain value, it is hard to improve, after 100 iterations, the accuracy of the 32p PSO remains the same as after 50 iterations.

The setting of PSO search space is important in getting results effectively, its takes much longer time to converge to optimal solution with bounds too large to explore.

5 Conclusions and future work

Based on the results, we can conclude that the proposed PSO based hyper-parameter tuning method can produce impressive results on CNN models in FL environment, the PSO algorithm is more efficient than grid search and GA, especially when the hyper-parameter search space and hyper-parameter dimension is large.

The performance of the proposed algorithm is affected by several factors, initialization, swarm scale, etc. Larger swarm scale leads to better performance since it requires more computation. Better initialization can be achieved by larger swarm scale when more particles are explored at the beginning. Choosing the right swarm scale can be crucial when computation resource is limited.

In future studies, the method could be implemented in different datasets since MNIST is a relatively simple one. More hyper-parameters associated with FL such as communication rounds and those associated with model like number of hidden layers are to be explored. Different deep learning models can also be implemented with the proposed method.

Acknowledgment

This work was supported in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2020B090921001 and in part by the National Natural Science Foundation of China under Grant 62036006 and Grant 61906146, and in part by the Fundamental Research Funds for the Central Universities under Grant JB210210.

References

[1] Kairouz P, McMahan H B, Avent B, et al. Advances and open problems in federated learning[J]. arXiv preprint arXiv:1912.04977, 2019.

[2] Qolomany B, Ahmad K, Al-Fuqaha A, et al. Particle swarm optimized federated learning for industrial IoT and smart city services. GLOBECOM 2020-2020 IEEE Global Communications Conference. IEEE, 2020: 1-6.

[3] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Artificial intelligence and statistics. PMLR, 2017: 1273-1282.

[4] Huang L, Yin Y, Fu Z, et al. LoAdaBoost: Loss-based AdaBoost federated machine learning with reduced computational complexity on IID and non-IID intensive care data[J]. Plos one, 2020, 15(4): e0230706.

[5] Ao Y C, Shi Y B, Zhang W, et al. Improved particle swarm optimization with adaptive inertia weight[J]. Journal of University of Electronic Science and technology of China, 2014, 43(6): 874-880.

[6] Bergstra J, Bardenet R, Bengio Y, et al. Algorithms for hyper-parameter optimization[J]. Advances in neural information processing systems, 2011, 24.

[7] Li X, Huang K, Yang W, et al. On the convergence of fedavg on non-iid data[J]. arXiv preprint arXiv:1907.02189, 2019.

[8] Shi P, Liang K, Han D, et al. A novel intelligent fault diagnosis method of rotating machinery based on deep learning and PSO-SVM[J]. Journal of Vibroengineering, 2017, 19(8): 5932-5946.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:39:12 UTC from IEEE Xplore. Restrictions apply.